Modernizing using-declarations

ISO/IEC JTC1 SC22 WG21, Evolution Working Group
P0195R0
Robert Haberlach (rh633@cam.ac.uk)
2015-12-23

*using-declaration*s should be able to cope with lists and pack expansions to allow flexible injection of names.

Motivation

With variadic templates having been introduced in C++11, many language constructs were updated to operate on pack expansions. Just as some situations require derivation from a pack of classes, some require introduction of a name from a pack of classes. Consider

```
template <typename... T>
struct Overloader : /* […] */ {
    // […]
};

template <typename... T>
constexpr auto make_overloader(T&&... t) {
    return Overloader<T...>{std::forward<T>(t)...};
}

int main() {
    auto o = make_overloader([] (auto const& a) {std::cout << a;},
                             [] (float f) {std::cout << std::setprecision(3) << f;});
}
```

The implementation of `Overloader` is verbose and inefficient; One has to recursively introduce overloads of `operator()`:

```
template <typename T, typename... Ts>
struct Overloader : T, Overloader<Ts...> {
    using T::operator();
    using Overloader<Ts...>::operator();
    // […]
};

template <typename T> struct Overloader<T> : T {
    using T::operator();
};
```

While it's possible to achieve logarithmic instantiation depth, the resulting implementation is needlessly complicated and still less efficient than the following, terse syntax:

```
template <typename... Ts>
struct Overloader : Ts... {
    using Ts::operator()...;
    // […]
};
```

Design decisions and impact

We propose that a *using-declaration* consist of a list of names, each called a *using-declarator*. The list shall not mix names of constructors with names designating other kinds of entities. As the proposed change is solely an enhancement, well-formed programs and their semantics are unaffected.

Proposed wording

Modify 3.3.2 [basic.scope.pdecl] ¶4 as indicated:

> The point of declaration of a *using-~~declaration~~declarator* that does not name a constructor is immediately after the *using-~~declaration~~declarator* (7.3.3).

Modify 3.4.3.1 [class.qual] ¶2 (2.2) as indicated:

> — in a *using-declarator* of a *using-declaration* (7.3.3) that is a *member-declaration*, if the name specified after the *nested-name-specifier* is the same as the *identifier* or the *simple-template-id*'s *template-name* in the last component of the *nested-name-specifier*,

Modify 7.3.3 [namespace.udecl] ¶1 as indicated:

> ~~A *using-declaration* introduces a set of declarations into the declarative region in which the *using-declaration*~~

~~appears.~~

using-declaration:
~~`typename`$_{opt}$ *nested-name-specifier unqualified-id*~~
`using` *using-declarator-list*`;`

using-declarator-list:
using-declarator`...`$_{opt}$
using-declarator-list`,` *using-declarator*`...`$_{opt}$

using-declarator:
`typename`$_{opt}$ *nested-name-specifier unqualified-id*

Each *using-declarator* in a *using-declaration*[98] introduces a set of declarations into the declarative region in which the *using-declaration* appears. The set of declarations introduced by the *using-~~declaration~~declarator* is found by performing qualified name lookup (3.4.3, 10.2) for the name in the *using-~~declaration~~declarator*, excluding functions that are hidden as described below. If the *using-~~declaration~~declarator* does not name a constructor, the *unqualified-id* is declared in the declarative region in which the *using-declaration* appears as a synonym for each declaration introduced by the *using-~~declaration~~declarator*. [*Note*: Only the specified name is so declared; specifying an enumeration name in a *using-declaration* does not declare its enumerators in the *using-declaration*'s declarative region. — *end note*] If ~~the~~a *using-~~declaration~~declarator* names a constructor, it declares that the class inherits the set of constructor declarations introduced by the *using-declaration* from the nominated base class.

---

98) A *using-declaration* with more than one *using-declarator*s is equivalent to a corresponding sequence of *using-declaration*s with one *using-declarator*.

Modify 7.3.3 [namespace.udecl] ¶3 as indicated:

In a *using-declaration* used as a *member-declaration*, ~~the~~each *using-declarator*'s *nested-name-specifier* shall name a base class of the class being defined. If ~~such a~~any *using-~~declaration~~declarator* names a constructor, all *using-declarator*s shall name constructors and ~~the *nested-name-specifier*~~their *nested-name-specifier*s shall name a direct base class of the class being defined.

Modify 7.3.3 [namespace.udecl] ¶10 as indicated:

[*Example:*

```
namespace A {
    int i;
}

namespace A1 {
    using A::i;
    using A::i, A::i; // OK: double declaration
}

struct B {
    int i;
};

struct X : B {
    using B::i;
    using B::i, B::i; // error: double member declaration
};
```

— *end example*]

Modify 7.3.3 [namespace.udecl] ¶19 as indicated:

If a *using-~~declaration~~declarator* uses the keyword `typename` and specifies a dependent name (14.6.2), the name introduced by the *using-declaration* is treated as a *typedef-name* (7.1.3).

Add a bullet (4.13) the list in 14.5.3 [temp.variadic] ¶4:

— In a *using-declaration* (7.3.3); the pattern is a *using-declarator*.

Acknowledgments

Thanks to Daveed Vandevoorde for assistance in preparing the wording.